

Group Values Load System



Index

Group Values Load System.....	1
INTRODUCTION	3
GETTING STARTED.....	4
GROUP VALUES	5
GVENTRYREFERENCE	10
GROUP VALUES TEMPLATE	12
LOADERS	13
EXPORT/IMPORT	18
GROUP VALUES EDITOR	18
CUSTOM DATA	22
GROUPVALUESWRAPPER.....	24
GROUP VALUES SIZE INSPECTOR	27
UIGVELEMENT	27
UTILS	30
INTERFACES IN GV	36
SETTINGSAPLIER/LISTENNERS	38
PROJECT SETTINGS	40
ENCRYPTION	42
SIMPLEGROUPVALUES.....	43
GROUP VALUES UTILITY	45
DEBUGGING	49
MIGRATIONS.....	50
BACKUPS.....	50
VERSIONING.....	51

INTRODUCTION

► About the tool

Welcome to the load system tool. A tool which will allow you to save data without using PlayerPrefs. It's flexible and can store a lot of serialized types.

► What can I store?

Basically every type. The tool can store in a defined generic structure all primitives.
And more!

► What can I use it for?

For everything! Your game data can be stored in group values. Your settings data can be stored too. It allows you to store all your game related data.

Have fun!

► Let's get started !

If you are eager to use this tool to it's fullest potential you can jump right into and create your first Group Values in Getting Started. What are you waiting for?

► Accessibility

This tool requires a medium to advanced knowledge about Unity and C#. It is recommended to read this documentation to understand the purpose and how this tool functions. It is advised that this tool can get a bit complicated in some aspects. However do not despair! If you know the workings of PlayerPrefs you can use SimpleGroupValues. This class works the same as PlayerPrefs with more supported types!

GETTING STARTED

► Create your first Group Values

Right click in your Resources folder and go to Create->LoadSystem->Group Values. A new GroupValues asset will be created. Open the editor and select your new asset. Add a field by clicking '+ Add Field' and then add a few entries inside it with '+ Add Entry'. Set some values and click Apply.

► Read and write values in code

No loaders needed yet. Create a new class to try. Just reference the SO directly:

```
[SerializeField] GroupValues myValues;

void Start()
{
    int speed = myValues.GetValue<int>("speed");
    myValues.SetValue<int>("speed", speed + 1);
}
```

► Make data persistent with LoaderMono

Add a LoaderMono component to any GameObject in your scene. Set the name to match your GroupValues asset. Press Save in the inspector to verify the JSON file is created. In code in the class you created before(or a new one), call LoadData() on Awake and SaveData() before closing the app:

```
[SerializeField] LoaderMono loader;
```

```

void Awake()
{
    loader.LoadData();
}

void OnApplicationQuit()
{
    loader.SaveData();
}

```

► Reference entries safely with GVEntryReference

Instead of using string keys in code, declare a GVEntryReference. In the inspector select your GroupValues, then the field, then the entry. No more risk of typos or broken references when renaming keys:

```

[SerializeField] GVEntryReference speedReference;

void Update()
{
    float speed = speedReference.Get<float>();
    speedReference.Set<float>(speed + 1);
}

```

► Your first build

In a build, save files are stored in persistentDataPath. Make sure LoadData() is called at startup and SaveData() is called before the app closes. If you want encryption go to Project Settings -> Load System and select an encryption method before building. If you want to add or remove entries after shipping read Migrations and Versioning.

GROUP VALUES

► What is it?

Group Values is the core of this tool. It is the basic element that allows data storage. It acts as an interface to provide structure and easy manipulation of data. In fact you can manually set values or add information going to the SO through the inspector (but that is not the smartest way to do it, see Group Values Editor).

► How to use it

Group values has a lot of different uses and cases. Most of all it is a scriptable object in which you can store almost any type of value. In scripts you can set and get different values that you can store later as persistent data.

► Structure

Although GroupValues is a flexible SO in which you can store anything this SO comes in a defined structure. The smallest element is called entry, in which you can store a value. All entries are defined within a field. All fields are confined in the GroupValue. That way you can store values in a certain area inside the SO.

GroupValues

```
|
|----->Field
|
|----->Entry(value)
```

Example:

Settings

```
|
|----->SoundSettings
|
|----->Mute(bool)
|----->MasterVolume(float)
```

► Creating Group Values

If you want to create new Group Values it is as simple as right-clicking on your project folder (in your Resources folder preferably) and then going to Create->LoadSystem->Group Values. A new instance of GV will be created.

► Adding a field

It is really simple. Just click on the bottom '+' button. A new field will be created.

► Adding an entry

Inside a field go click the bottom '+' button. A pop up will appear to make you choose a type for that entry value. After you selected one you can edit the value and change the entry name.

IMPORTANT NOTE: entry names must be different.

► GetValue<T>

When you want to get a certain type of value you will ask for the name of value that has the same type. So for example, if you want an int value with name "speed" to your variable topSpeed the correct way to write it is:

```
GroupValues myValues;  
  
int speed = myValues.GetValue<int>("speed");
```

► Exceptions in GetValue<T>

If the key does not exist it will throw an error. This is important to know because it can crash the build. It will also throw an error if an invalid cast is provided. To prevent crashes it is recommended reading about [GVEntryReference](#) .

► SetValue<T>

In the same way you can get a value out of the class you can store it as well in the scriptable object. Following the same example if you want it to assign a value and then store it the correct way to do it is:

GroupValues myValues;

```
int speed = 2;
myValues.SetValue<int>("speed", speed);
```

► Exceptions in SetValue<T>

If the key does not exist it will throw an error. This is important to know because it can crash the build. It will also throw an error if an invalid cast is provided. To prevent crashes it is recommended reading about [GVEntryReference](#) .

► TryAddEntry

You can dynamically add entries to any Group Values. It is not recommended but you can do it if you want. It needs a [GVEntry](#) and a field name. Example:

```
GroupValues gv;
string valueName = "DynamicBool";

GVEntry entry = new GVEntry()
{
    name = valueName,
    type = VALUE_TYPE.BOOL,
    customTypeName = "",
    value = new BoolGVValue()
};
```

```
entry.value.SetValue(false);

gv.TryAddEntry("TestingValues", entry);
```

Or if you want to add a Custom class:

```
GroupValues gv;
    string valueName = "DynamicBool";

    GVEntry entry = new GVEntry()
    {
        name = valueName,
        type = VALUE_TYPE.CUSTOM,
        customTypeName = "PlayerData",
        value = new StringGVValue()
    };

    gv.TryAddEntry("TestingValues", entry);
```

► Exceptions

If the entry already exists it will log a warning. PLEASE use in value supported GV types. If not used properly unity can get confused with serialization.

► TryRemoveEntry

The same way you can create entries dynamically you can remove them. Example:

```
GroupValues gv;

    string valueName = "DynamicBool";

    gv.TryRemoveEntry(valueName);
```

► Exceptions

Warnings will be logged if entry name is not in GV.

► Other supported types

If you want to add any other type than the ones shown in the pop up entry you can add custom data. For more information read Custom.

► DataPersistence

It is very important for the user to know that this methods store the values inside the SO(Scriptable Object) but it does not mean persistence in your data stored. For more information read Loaders.

► Performance

Underneath Group Values there is a Dictionary that caches all entries making all data accessing O(1). This means that Get and Set are really fast (almost instantly). You can use bench marking to test it yourself.

=====

GVENTRYREFERENCE

=====

► What is it?

Do you want to prevent build crashes? Are you terrified of using a string key that you may later change and forget it was being referenced by a script and suddenly everything is on fire? Do not worry, GVEntryReference solves that problem.

► How do I use it?

Very simple, you need a class that needs an access to a certain type of value with a certain key. Declare GVEntryReference in your script. Then in the inspector go to GVEntry->SelectGroupValues->SelectField->SelectEntry. Now you have complete access to that entry. No more key using.

► Get<T>()

Let's say you want to get your value, it is as simple as writting something like this:

```
[SerializeField] GVEntryReference speedReference;  
  
[SerializeField] float speed;  
  
speed = speedReference.Get<float>();
```

► Set<T>()

Now let's say you want to set your value, it is as simple as writting something like this:

```
[SerializeField] GVEntryReference speedReference;  
[SerializeField] float speed;  
  
speed = 5;  
speedReference.Set<float>(speed);
```

► Key aspects

Whenever a key is changed, in the inspector a warning inside a box will be triggered. Also a warning message may be triggered on the console. Do not worry. It will auto-assign again. Although manual reassignment is strongly recommended to avoid errors. Also, Loaders are still needed if you want persistence in your data. Setters only apply to referenced GroupValues.

GROUP VALUES TEMPLATE

► What is it?

Templates are your Group Values readonly data. When building your game the preprocessor will set your data to default values. But what if you want certain entries with different default values? Templates provide a solution to fix a default value to a Group Values reference.

► How to use it?

Templates need a Group Values reference. Once referenced it can access to the entry values and set a default value for them. You do not need further set up. The preprocessor will automatically set those values as default to the referenced Group Values.

► How do I enable it?

Really simple, go to Project Settings->Group Values. Set Use Templates to true.

► SetDefaultValuesInTemplate

Let's say you are creating a fresh template. First of all you need to reference any type of Group Values. Then you have to call this method to copy all fields and entries in order to create default values for your referenced SO. For example:

```
GroupValuesTemplate gvT;  
  
GroupValues gv;  
  
gvT.groupValuesReference = gv;  
gvT.SetDefaultValuesInTemplate();//all fields and entries are  
copied in the template
```

Although you can do this, it is not recommended as you are being provided with a window that makes your life easier.

► SetDefaultValuesInSO

Once you have set a copy inside your template you can then adjust values and set them in your Group Values. This practice is not recommended as the preprocessor does the job automatically.

```
//do as in previous example
...//change data
gvT.SetDefaultValuesInSO();//all fields and entries are set in
the referenced Group Values
```

Again, this is not recommended. It is not recommended to call this manually.

► Key aspects

Inside a Template you should not change anything other than values. This means that changing a field name or entry name can lead errors or to unexpected behaviour in worst cases. The window editor will not let you commit these actions. For more information, read Group Values Editor.

=====

LOADERS

=====

► What are they?

Group Values is just the tip of the iceberg. All data stored in them is not persistent :(.This means that games can't store data using only Group Values. However, loaders are here to solve this problem.

► Where are the files stored?

That is a really good question. When using the editor .json files will be stored in the same path as the GV they reference. They will be hidden though. When using the build they will be stored in the PersistentData path (%USERPROFILE%\AppData\LocalLow).

► ALoader

This is a C# pure class. It takes a GroupValues reference and can convert it to a json file. You can set your GroupValues name and a path to start saving your data. However, you cannot use it as a component.

► Structure

ALoader contains the GV to save/load and acts as the controller of a single GV.

This way you only interact with a loader whenever you want to manage data. This way the loader rules over GroupValues and provides an abstraction layer.

ALoader

|

|

|->LoadValues/SaveValues/Get/Save----GroupValues

|->LoadValues/SaveValues-----JSON

► Attributes

ALoader has different attributes, but most importantly it contains a reference to its assigned Group Values. Inside the class you will see:

```
//
```

```
//path inside Resources folder
```

```

        [SerializeField] protected string soPath =
"Assets/Resources/LoadSystem/SavedFiles/";
        //Name without extension
        [SerializeField] protected string baseName = "Game";

        protected string soName => baseName + ".asset";
        protected string jsonFileName => "." + baseName + ".json";

        //Project settings
        [ReadOnly][SerializeField] EncryptionMethod encryptionMethod =
EncryptionMethod.None;
        [ReadOnly][SerializeField] string password = "";

        //The core, GroupValues reference
        [SerializeField]
        [ExposedScriptableObject]
        protected GroupValues values;
        //

```

► Methods

ALoader provides the same methods as Group Values (as it provides an additional abstraction layer to the SO). This means that it comes with Get and Set and work the same.

```

ALoader loader;//loader has speed set as 5
int speed = 0;//initialize variable
speed = loader.GetValue<int>("speed");
speed += 1;
loader.SetValue<int>("speed", speed);// value is set to 6

```

► LoadData

ALoader can load data from a json for you. If the path and name are set up correctly it will read the data from the json file and pour it inside the referenced GroupValues. It also returns GV loaded.

```
//before reading make sure at the start to have synchronized data

ALoader loader;
loader.LoadData();

int speed = loader.GetValue<int>("speed");
```

There is also async loading:

```
ALoader loader;
await loader.LoadValuesAsync();
```

► Exceptions in LoadData

Not assigning a name or name that does not exist in Resources folder will not find GV and will log an error message every time you load data.

► SaveData

ALoader can save data to a json as well. If the set up is correct. It will perform the action without any problems.

```
//you can do this before closing app for example

ALoader loader;
int speed = 5;
loader.SetValue<int>("speed", speed);
loader.SaveData();
```

There is also async saving:

```
ALoader loader;  
  
await loader.SaveValuesAsync();
```

► Exceptions in SaveData

Not assigning a name or name that does not exist in Resources folder will not find GV and will log an error message every time you save data.

► LoaderMono

LoaderMono comes as your useful component and implements ALoader to save the day(and your data). This component allows you to set, get, load and save data in runtime as ALoader does. Yippie!

It works the same as ALoader as it provides an abstraction layer. As mentioned before you can set your GroupValues name and click on AutoFindPathByName to have everything set up.

► Buttons in LoaderMono?

These buttons do the as what their name implies. You can save and load through these buttons or set to default without using Group Values Editor or scripting.

► Key Aspects

It is up to the user to decide when to load or save data.

In project settings you can set the encryption method and change the password(For more information read Encryption or Project Settings).

► Other exceptions

Loaders will throw exceptions when path is not correctly set, Group Values is not found or when there is no Group Values to save. Be careful and keep these things in mind.

=====

EXPORT/IMPORT

=====

► What is it?

Have you ever tried to move a file .asset to a different project? Even though the SO script is in the project unity will remove all data. This tool comes with a solution. You can export your GroupValues.asset to a json, xml or csv file to then import such file in other projects. This means you can recover Group Values and reuse them for future projects! Yay!

► Export

By selecting Tools->LoadSystem->Export Group Values you will open the export window. Select from the Assets folder any type of Group Values. Then select the file format, click 'Export' and save it anywhere.

► Import

By selecting Tools->LoadSystem->Import Group Values you will open the import window. Select your file with the same file extension. Then select somewhere inside Assets to save the new Group Values (it is recommended to save it within a Resources folder).

=====

GROUP VALUES EDITOR

=====

► What is it?

This tool allows an easier way to add more entries or more fields, make changes to entry values and much more. It is the best way to manage your Group Values as when applying changes it will also save it to json.

► How do I open it?

Go to Tools->LoadSystem->Group Values Editor. Do not be overwhelmed by the amount of buttons we will cover them one by one.

► GroupValues/Templates

The first thing you will see is that GroupValues is in color blue. This is a toolbar that tells you that you are currently working on Group Values. You can change it to edit Templates too.

► Open Project Settings

This button is pretty much self-explanatory. It opens the LoadSystem project settings.

► Refresh List

Just in case your freshly created GV(Group Values) does not show on the list below you can click on this button to make sure it appears.

► Selected Asset

Inside the list it provides it sets the GV that you will work on.

► Working on Group Values

All buttons stated appear for both workplaces. However similarities end here and each class possesses its own buttons with equivalent behaviour.

► Create Group Values

It creates a new GV SO wherever you want.

► Delete Selected

Deletes the current working GV

► Reset All

Sets all GV values to default. It does not keep in mind templates.

► View Template Values

It is a toggle that when activated allows you to see the values that the template will set on your working GV.

► Copy From Current

Copies the current working GV and lets you save it wherever you want

► Apply

This is the most important button as it saves all changes and creates the json file.

► Reset to Defaults

Sets current working GV to default values

► Working on Templates

As stated before Templates have buttons with similar behaviour.

► Create Template

It creates a new Template wherever you want.

► Delete Selected

Deletes current working template.

► Reset All

Resets all Templates to default values.

► Template from GV

This has the same behaviour as the method `SetDefaultValuesInTemplate`. It copies the referenced GV fields and entries.

► Template to GV

This has the same behaviour as the method `SetDefaultValuesInSO`. It sets the referenced GV fields and entries with the template values.

► Apply

Saves changes in the template.

► Reset To Defaults

Sets all current working Template values to default.

CUSTOM DATA

► What is it?

Do you feel like you don't have enough types to save? Do you want to make your own clean custom sets of data instead of making a thousand entries? The type Custom has you covered. With it you can set your class as custom and chuck it inside GV.

► How does it work?

As everything in this tool it is really simple. You only have to add the property [CustomSettingData("Name")] on top of your class declaration. For example:

```
//
[CustomSettingData("PlayerStats")][Serializable] //Serializable
property is a must
public class MyPlayerStats
{
    public Vector3 position;
    public int speed;
}
//
```

Then it will appear in the registry. Create a new entry and set your custom data class.

► Set and Get Custom Data

If you want to set or get a custom data class you need to save or load the entire class. For example using GVEntryReference:

```
MyPlayerStats stats;

GVEntryReference statsReference;

//First read loaded data
stats = statsReference.Get<MyPlayerStats>();

//Then change data and save
stats.speed = 5;
statsReference.Set<MyPlayerStats>(stats);
```

► Custom Data inside Custom Data?

The answer is yes. You can add more than one custom data level.

For example you can do this:

```
[CustomSettingData("GameStats")][Serializable]
public class MyGameStats
{
    public int time;
    public string CurrentChapter;

    public MyPlayerStats playerStats; //MyPlayerStats is custom too :)
}
```

HOWEVER, DO NOT SELF REFERENCE OR HAVE TWO CLASSES REFERENCE EACH OTHER.

If you want to avoid stack overflow I recommend thinking very well before adding custom data inside custom data. User is advised.

► Key aspects

It also allows Quaternion, Color, Vector4, Bounds and AnimationCurve serialization among others. But most importantly it allows Lists made of primitive types. It will not serialize anything with a gameObject reference.

Variables inside CustomData can also be referenced with GVEntryReference. When referencing CustomData an extra path will appear to drill down the class.

=====

GROUPVALUESWRAPPER

=====

► What is it?

Do you want all the features that Custom Data provides but you do not want to add the tag [CustomSettingData("NameofCustom")] to every single class you need to save? GroupValuesWrapper has you covered.

► How do I use it?

It is quite simple actually. First of all your class must be serializable. Add [Serializable] on top of your class if the tag does not exist. For example:

```
//  
    [Serializable] //Serializable property is a must  
    public class MyPlayerStats  
    {  
        public Vector3 position;  
        public int speed;  
    }  
//
```

Then declare wrapper variable in your mono component:

```
GroupValuesWrapper<MyPlayerStats> stats;
```

This will add it to your Custom collection as another valid Custom entry.

► Initializing the Wrapper

The wrapper in order to be initialized properly needs a GroupValues reference and a key.

You can skip initialization as the wrapper has static Load and Save. But if the wrapper has to be declared somewhere in order to access a wrapped class. For example:

```
public GroupValuesWrapper<MyPlayerStats> stats;  
public GroupValues gv;  
public string key;  
  
public void Start()  
{  
    stats = new GroupValuesWrapper<MyPlayerStats>(gv, key);  
}
```

► Get and Set using a Wrapper

With a similar behaviour as Custom Data the entire class is loaded in the referenced entry. Here are two different ways to use the wrapper:

Initializing the wrapper.

```
public MyPlayerStats playerStats;  
  
public GroupValuesWrapper<MyPlayerStats> stats;  
public GVEntryReference reference;  
  
public void OnEnable()  
{  
    stats = new  
GroupValuesWrapper<MyPlayerStats>(reference.GroupValues,  
    reference.EntryKey);  
}  
  
[Button("SetStats")]  
public void SetStats()
```

```

{

    playerStats = stats.Load();

    playerStats.speed = 5;

    stats.Save(playerStats);

}

```

Using static methods.

```

public MyPlayerStats playerStats;

    //stats must be declared on this script or elsewhere but a wrapper of
    MyPlayerStats type must be somewhere.
    public GroupValuesWrapper<MyPlayerStats> stats;
    public GVEntryReference reference;

    [Button("SetStats")]
    public void SetStats()
    {

        playerStats =
GroupValuesWrapper<MyPlayerStats>.Load(reference.GroupValues,
        reference.EntryKey);

        playerStats.speed = 5;

        GroupValuesWrapper<MyPlayerStats>.Save(reference.GroupValues,
        reference.EntryKey, playerStats);

    }

```

Both ways are valid. However, initializing the wrapper requires less writing when trying to access data.

► Key aspects

The entry for the wrapper will appear as an entry in Group Values once declared on your script. Then create a new entry with Custom type and select the wrapped class.

As wrappers are grouped with Custom Data the variables within them can also be referenced with GVEntryReference.

=====

GROUP VALUES SIZE INSPECTOR

=====

► What is it?

If you are curious about how much size do all your GroupValues take this tool gives you exactly that. It shows how much space does the .asset and the .json take and sums it all. Go to Tools/LoadSystem/GroupValues Size Inspector.

=====

UIGVELEMENT

=====

► What is it?

Watching your values change in the inspector is really nice. However, people that play your game want to see those values INSIDE the game, **** the inspector. That is why UIGVElement is presented to you as a solution to see the values changed in runtime in UI elements. It is a component that you can add to a UI element(toggle, drawer,slider or input field) and loads and saves an entry value perfectly.

► How does it work?

It is a bit tricky. However this script has been simplified to require 0 user modification. There are two methods in which ui can access Group Values.

The first one is through a window provided by this tool and the second one needs a little bit of programming. Both are valid and window usage is encouraged to prevent errors.

For the second method in the conceptual field you need a manager (a `SettingsManager` for example) that registers itself in a service locator. Once registered, each ui element can access the locator and ask for the registered instance. With that instance it can Get and Set values as a normal GV.

► First Method

Open the window in Tools->LoadSystem->UI Settings Elements.

Yet another window, I know. But this one is as simple as everything provided by this tool.

► *Window Set Up*

This window creates ui elements based on prefabs. Some examples are provided to you. Click on Prefab Defaults, it will show a list of different types of UI elements and their prefab assigned. This way you can assign your own prefabs or use the provided examples. Prefabs will be saved for further use.

► *First Step-Create a UIGVElement*

Inside the window click on the '+' button. Set a name to the element. Pick an entry and the value and type of ui element will be auto-assigned. The prefab will also be auto-assigned based on the type of UI. If you prefer other type of prefab you can manually change it. Everything can be manually changed. Assign a Canvas parent Then click on create.

► Second Method

I know that this method might sound bizarre so let me guide you with the steps needed to link ui and GV.

► *First Step-Register your SettingsManager*

Your settings manager must implement `IValuesContainer` and register itself on the static class `ValuesContainerLocator`. For example, this set up should do:

```
public class SettingsManager : MonoBehaviour, ISettingsContainer
{
    public GroupValues settingsValues;
    [SerializeField] List<UIGVElement> uiElements;

    public void Awake()
    {
        settingsValues = GetComponent<LoaderMono>().LoadValues();
        SettingsContainerLocator.Register(this);
    }
    public void SetValue<T>(string key, T value)
    {
        if (settingsValues == null) return;
        settingsValues.SetValue<T>(key, value);
    }
    public T GetValue<T>(string key)
    {
        if (settingsValues == null) return null;
        return settingsValues.GetValue<T>(key);
    }
}
```

► Second Step-Reference value

Then you can access your list of `UIGVElement` and assign values in your manager. Following the same example you can set:

```
public class SettingsManager : MonoBehaviour, ISettingsContainer
{
    public GroupValues settingsValues;
    [SerializeField] List<UIGVElement> uiElements;
    //... interface functions

    //Initialize interface with stored values in settingsValues
    public void LoadData()
```

```

    {
        foreach (var uiEl in uiElements)
        {
            uiEl.LoadData();
        }
    }

    //When closing ui save as json with loader
    OnCloseUI()
    {
        GetComponent<LoaderMono>().SaveData();
    }
}

```

This gives you more control about when to load data(default Behaviour in UIGVElement must be disabled). Following this pattern you can create your own interfaces and locators for other type of cases.

► Key Aspects

If your UIGVElement is created and left with default values it auto-detects the ui element and on enable it will load the value. IMPORTANT NOTE: Key value must be the same as in GV. Also if no key value is provided it will try with gameobject name. In addition, key value must be within the entry name of the same GV referenced in the SettingsManager.

UTILS

► Button

```
[Button("Button")]
```

Instead of using context menu to test functions on the inspector button provides a button that can execute parameterless methods.

For example:

```
[Button("Hello World")]

public void Log()//methods must be public in order to show the button
{
    Debug.Log("Hello World");
}
```

► Exposed Scriptable Object

```
[ExposedScriptableObject]
```

Allows you to see all attributes from an Scriptable Object inside a MonoBehaviour. It also allows writing on it.

For example to show GroupValues ALoader uses:

```
[SerializeField]

[ExposedScriptableObject]
protected GroupValues values;
```

► Unique Reference

```
[UniqueReference]
```

This is a special type of tag. It constrains an object to only be referenced by a single class.

For example, this is used in Templates:

```
[UniqueReference]

public GroupValues groupValuesReference;
```

► Show If

```
[Showif("boolIstrue")]
```

It only shows a variable if a condition is met. It also works for more than one condition.

```
[Showif("bool1Istrue","bool2Istrue")]
```

The more used case scenario I personally use is:

```
[Header("Debugging")]
[SerializeField]
debug=true;

[SerializeField]
debugGame=true;

[SerializeField]
debugScene=false;

[SerializeField]
[ShowIf("debug","debugGame")]//This will appear on Inspector
//debug and debugGame = true

debugGameSpeed=5;

[SerializeField]
[ShowIf("debug","debugScene")]//This will not appear on Inspector
//debug and debugScene = false

debugTickSceneSpeed=10;
```

► ReadOnly

```
[ReadOnly]
```

Makes a serialized field as a read only value. You can not edit that value. For example, if you want to see the value of a variable you can set it as this:

```
...
```

```
[ReadOnly] public float tickTimer=0;
```

```
...
```

► CustomLabel

```
[CustomLabel("Overriden propfield name")]  
[CustomLabel]
```

It allows you to override a property name instead of using the one it has assigned. Adding it without arguments hides the name completely.

For example:

```
[SerializeField,CustomLabel("SPEED")]float speed; //the inspector  
will show SPEED  
  
[SerializeField,CustomLabel]float maxSpeed;//the inspector will NOT  
show a name assigned to maxSpeed
```

► CustomData attributes

Inside a custom class you can set special attributes depending on what behaviour will have your values.

► SaveAs

```
[SaveAs("OtherName")]
```

Works the same as CustomLabel but for variables inside custom data.

```
[SaveAs("SPEED")]float speed;
```

► DontSave

```
[DontSave]
```

If for whatever reason you do not want a value to be saved because it is temporal you can mark it with this attribute.

```
[DontSave] float speedTimer=0;
```

► GVReadOnly

```
[GVReadOnly]
```

Makes a field read-only in both the inspector and via GVEntryReference. The value is still serialized and loaded normally — it is only protected from manual edits.

```
[GVReadOnly] public string readonlyStr= "This is read only";
```

► GVMin and GVMax

```
[GVMin(n)] [GVMax(n)]
```

Adds a min or max restriction to numeric values. It ONLY works on scalars.

► GVRange

```
[GVRange(min,max)]
```

Adds a range restriction to numeric values. It ONLY works on scalars.

► WriteOnce

```
[WriteOnce]
```

It only writes a value one time, can be useful for storing a once per app open value. It can be useful for storing keys or in-game passwords randomly generated. Click on the R button to reset and be able to write once more.

```
[WriteOnce]  
float puzzlepassword;
```

► WriteN

```
[WriteN(n)]
```

It can write a value n times, if for whatever reason you need this it is provided. Click on the R button to reset and be able to write once more.

```
[WriteN(3)]  
int countDown;  
public int __countDownWriteCount; // companion field required to  
track write count
```

=====

INTERFACES IN GV

► What are they?

The interfaces provided by this tool are examples of how your scripts can interact with Group Values.

► ISettingsContainer

This interface was first mentioned in `UIGVElement` and is the perfect example when using managers and GV. Inside of it you can find:

```
public void SubscribeToSettingsChange(Action onChange);  
  
public T GetValue<T>(string key);  
public void SetValue<T>(string key, T value);
```

And below the interface an static class with similar name:

```
public static class SettingsContainerLocator
```

To make your manager work register your interface on Awake.

```
SettingsContainerLocator.Register(this); //Manager
```

Now classes that need your manager to find their respective values can find it easily with:

```
SettingsContainerLocator.Get() //Smaller class
```

Now when asking for a value the script does not need a GV reference and it is decoupled.

```
float
speed=SettingsContainerLocator.Get().GetValue<float>("speed");//Smaller
class
```

You can use this piece of code to set up a manager:

```
public class ManagerSettings : MonoBehaviour, ISettingsContainer
{
    public GroupValues gv;
    public UnityEvent onSettingsChange;

    public void Awake()
    {
        SettingsContainerLocator.Register(this);
    }

    public void SubscribeToSettingsChange(Action onChange);
    {
        onSettingsChange.AddListener(onChange);
    }

    public T GetValue<T>(string key)
    {
        return gv.GetValue<T>(key);
    }

    public void SetValue<T>(string key,T value)
    {
        gv.SetValue<T>(key,value);
        onSettingsChange.Invoke();
        //GV has been changed, notify all observers
    }
}
```

This interface gives guidance on what your code architecture can look like when using GV. This way is clean and avoids referencing GV on every script.

► ILoaderUser

This interface can apply to all those scripts that need a certain value from a GV.

It comes with:

```
public void SubscribeToValuesChange();  
    public void OnValuesChange();
```

Inside of the first method you can subscribe to UnityEvents that are triggered when a value is changed. On the second one you decide what values are being read when values change.

This means that ILoaderUser and ISettingsContainer can be used together.

You can use this piece of code to use a manager with this type of interface:

```
public class LoadSpeed : ILoaderUser  
{  
    ISettingsContainer container;  
    public float speed;  
  
    public void Init()  
    {  
        container = SettingsContainerLocator.Get();  
  
        if (container != null)  
            container.SubscribeToSettingsChange(OnValuesChange);  
        //Set method for when UnityEvent is invoked  
    }  
  
    public void OnValuesChange()  
    {  
        speed = container.GetValue<T>("speed");  
    }  
}
```

=====

SETTINGSAPPLIER/LISTENNERS

=====

► What are they?

These are examples of how you can use GV and having some listeners that react when values have changed.

► SoundSettings

This script provides a full fledged example of what you can achieve with GV and a good design in architecture. Go check it out in LoadSystem/Examples

► ViewportSettings

This script also provides a full fledged example. Go check it out in LoadSystem/Examples

► Key aspects

As you can see, these scripts act as observers or listeners however you want to call them. When being initialized they look for a container instance. It is seen on:

```
SubscribeToValuesChange();
```

If a container is found, then it subscribes to its UnityEvent.

```
container = SettingsContainerLocator.Get();

    if (container != null)//if there is a settingsContainer,
subscribe OnValuesChange method
        container.SubscribeToSettingsChange(OnValuesChange);
```

Then, whenever a UnityEvent is invoked these listeners will pick their own values and apply changes.

This means that this tool gives more purpose than just storing data. It makes it easier to pick up values stored and avoids spaghetti code.

PROJECT SETTINGS

► What is it?

This tool provides all in all a lot of functionality. But as a developer you do not require all tools at all times. That is why you can go to Project Settings and change a lot of behaviours.

► Group Values

In here you can change some settings

► *Save Subfolder*

Allows you to save in a subfolder in persistent path all json files.

► *Use backups*

Creates a backup copy of json file. If original file is corrupted or not found a new GV will be made by this backup and it will make a copy of this backup as the new original file. Read more in [Backups](#).

► *Use Templates*

Apply created templates in project over referenced Group Values.

► *Encryption Method*

Allows selection between three different methods of encryption. Aes 256_Hmac Sha 256 is the safest among the three. You can always have none if you do not mind keeping your json files safe. This encryption methods are global and all loaders are updated when changes are made.

► *Password Salt*

This password is what encrypts and deciphers json files if a method other than none is used. Pressing Generate New Salt creates a new password.

► *Enable Info Logs*

Adds to scripting symbols the symbol LOG_LOADSYSTEM to stop annoying loggings from the tool. You can use it as well.

► *Show Help Button*

If you did not notice there are buttons with ? that lead you to this documentation. If you want to disable those buttons click on the toggle.

► *Debug Overlay*

It contains all data in reference to debugging Group Values. For more information read about Debugging.

► *Enable Overlay*

Allows using the Debug Overlay Window.

► *Debug Key*

Which key is pressed in order to make the window appear. IMPORTANT NOTE: If using the new Input System keep in mind to use in Active Input Handling "Both". Otherwise the key will not work.

► *Edit Mode*

Allows user to edit values. It is very useful depending on the situation.

► *Freeze Time on Open*

Sets Time.timeScale to 0 when opening the window. Keep it in mind when using it as it can mess with time if other script is also messing with timeScale.

► Theme

Not the most important aspect inside the tool but keeps windows pretty and organized.

There are four default themes to choose from. Minimal uses the same colors as unity. Although my favourite is OceanBlue.

There are buttons to edit the current theme, create a new one or regenerate defaults (if you have erased them, you monster). They come with a bunch of settings that you can adjust if you want a deep level of customization. Also text size is customizable.

=====

ENCRYPTION

=====

► What is it?

Are you afraid that your data is not safe by exposing all your GV information? Are you afraid that your data can be modified by clients? You are right ! This is why this tool ensures safety by providing different types of encryption.

► How do I start applying encryption to files?

Really easy, go to Project Settings->Group Values . Then select your preferred encryption method. A key will be randomly generated.

► How does it work?

The final encryption key is derived by combining two elements: a project-wide password salt (set in Project Settings) and a device key unique to each installation. The device key is a randomly generated file stored in persistentDataPath the first time the game runs. From that point on it is always reused, making the encryption unique per user and per project.

This means two players with the same build cannot read each other's save files, and a save file cannot be moved to another device.

IMPORTANT NOTE: on PC the device key persists across reinstalls as long as persistentDataPath is not manually deleted. On mobile platforms, uninstalling the app deletes persistentDataPath along with the device key — if the player reinstalls, a new key is generated and any previously encrypted save files become unreadable.

=====

SIMPLEGROUPVALUES

=====

► What is it?

As its name suggests it is a more user friendly class to save all your data. If you are already using Group Values or intend to use it I recommend skipping this.

► How do I use it?

It is really simple as it works exactly as PlayerPrefs. You do not have to create a Scriptable Object nor do you have to create entries. If you are not familiarized with it here are some examples.

► Save values with Set<T>

You only need a key and a value. Do not fear generics! If you want to set a value for example, speed you only have to write this line of code:

```
float speed = 5;
SimpleGroupValues.Set<float>("speed", speed);
```

► Get values with Get<T>

For this, you only need a key. As simple as this:

```
float speed = SimpleGroupValues.Get<float>("speed");
```

► Is there a way to watch all saved data?

Yes, absolutely there is a way to watch all your saved data (Unlike PlayerPrefs (ಠ_ಠ)). Go to Tools->LoadSystem->SimpleGroupValues Viewer.

► Key aspects

SimpleGroupValues provides a ready to go solution for storing data in with way more types than PlayerPrefs and gives the user the option to see their created data. However, it does not incorporate all the advantages that GroupValues brings to the table. It can not use Templates and do not give the user the control in general that Group Values gives. Group Values allows you to make a more structured data storage and is way more better.

► Exceptions

Keep in mind what entry key and what value you are requesting when setting or getting a value. You CAN NOT get a value you HAVE NOT set.

GROUP VALUES UTILITY

► What is it?

This is a tool that you may require when working with bigger data amounts. It can provide lists of entries of the same type. It can count how many type entries there are, return a list of keys for a value type, make sums, calculate average (if type is numeric).

► Methods

Here is a list of all the methods that come with this static class.

These can be used to calculate statistics about your stored data.

Most method take always as argument GroupValues. You can also pass the field name if you only need to know the result from a certain field.

► *GetAll<T>*

Returns a list of type T. Example:

```
List<int>speeds=GroupValuesUtility.GetAll<int>(gv);
```

► *CountAll<T>*

Counts all entries of type type. Returns an integer.Example:

```
int speedCount=GroupValuesUtility.CountAll<int>(gv);
```

► SumAll<T>

Sums all numeric type data. Can sum vectors up to Vector3 (VectorsInt excluded). Example:

```
float sumSpeeds=GroupValuesUtility.SumAll<float>(gv);
```

► Max<T>

Gets the max numeric type data. Example:

```
float MaxSpeed = GroupValuesUtility.Max<float>(gv);
```

► Min<T>

Gets the max numeric type data. Example:

```
float MinSpeed = GroupValuesUtility.Min<float>(gv);
```

► Average<T>

Calculates the numeric type average. Can calculate average of vectors up to Vector3 (VectorsInt excluded). Example:

```
Vector3 averagePoints=GroupValuesUtility.Average<Vector3>(gv);
```

► Variance<T>

Calculates the numeric type Variance. Can calculate variance of vectors up to Vector3 (VectorsInt excluded). Example:

```
float speedVariance=GroupValuesUtility.Variance<float>(gv);
```

► StdDev<T>

Returns the square root of Variance<T>. Example:

```
float speedStd=GroupValuesUtility.StdDev<float>(gv);
```

► CalculatePercentages<T>

Calculates the percentages of numeric types.

For example if you have three different entries of type int. Entry1=300, Entry2=70 and Entry3=20. In total they sum 390. So for Entry1 its percentage is $300/390 * 100$ and so on.

Use FormatPercentages to see full information. Example:

```
GroupValues gv;

var typedist = GroupValuesUtility.CalculatePercentages(gv);

if (typedist.Count > 0)
{
    Debug.Log($"Type dist    →
{GroupValuesUtility.FormatPercentages(gv)}");
}
```

► MostUsedWords

Returns most used words in your string entries. Use `FormatMostUsedWords` to see full information. Example:

```
var strings = GroupValuesUtility.GetAll<string>(gv);
if (strings.Count > 0)
    Debug.Log($"Words    →
{GroupValuesUtility.FormatMostUsedWords(gv, topN: 5)}");
```

► TypeDistribution

Returns the percentages of all entry types used in your `GroupValues`. Use `FormatTypeDistribution` to see full information. Example:

```
var typedist = GroupValuesUtility.TypeDistribution(gv);

if (typedist.Count > 0)
{
    Debug.Log($"Type dist    →
{GroupValuesUtility.FormatTypeDistribution(gv)}");
}
```

► Median<T>

Returns the median of numeric types. Can calculate median of vectors up to `Vector3` (`VectorsInt` excluded). Example:

```
float medianSpeed = GroupValuesUtility.Median<float>(gv);
```

► Normalize<T>

Returns a normalized list of values between 0 and 1 (as floats) based on min and max value. Can calculate normalization of vectors up to Vector3 (VectorsInt excluded).

Example:

```
List<float> speedsNorm=GroupValuesUtility.Normalize<float>(gv);
```

DEBUGGING

► How do I debug?

Are you having troubles because some value is being changed? Maybe you want to set a certain value without using the inspector. Maybe you are inside the build and don't want to show all data in the ui, but you want to see certain values being changed. How about you check out the debugging window.

► How does it work?

By enabling it in project settings and pressing a key (F1 by default) a window will pop up. In there you can select any GroupValues you wish to see or modify values.

► How do I enable the debugging window?

It is fairly simple, go to Project Settings->Group Values. You also need to activate Development Build on Build Settings. Go to Enable Overlay and set it to true. You can set the key you prefer. Whether you want to have values only be read. In addition you can freeze time when the window is open.

MIGRATIONS

► What is it?

What if you want a scalable system when adding or removing entries while keeping user data untouched? Migrating correctly your GroupValues can answer to that question. But first you need to know how it works. Keep in mind that migration is only needed once you have created your first build.

► How does it work?

Inside Group Values Editor when adding or removing an entry or field a suggested bump message will appear. Accept suggested change or manually change it.

► What happens next?

User values stored in the build are preserved and new or deleted data is synchronized.

► Key aspects

KEEP IN MIND to update (Template from GV) your templates as well as they can override your changes.

BACKUPS

► What is it?

The backup system protects player save data from corruption. When enabled, every time data is saved a backup copy is written first, then the main file is written. This means there is always a valid previous version available.

► How does it work?

On load, the system tries the main file first. If it is corrupted or missing, it falls back to the backup automatically and restores the main file from it. If both are missing or corrupted, default values from the SO are used and both files are recreated.

Save order:

1. Write backup (.backup.json)
2. Write main (.json)

This means if the game crashes mid-save, the backup always holds the last valid state.

► How do I enable it?

Go to Project Settings -> Load System and enable Use Backups. All LoaderMono components in the project will be updated automatically.

=====

VERSIONING

=====

► What is it?

Each GroupValues has a version number (major.minor.patch and an optional label). This version is saved in the JSON file and compared against the SO version at load time. If they differ, a migration is triggered automatically to merge player data with the new structure.

► How do I change the version?

At the bottom of the Group Values Editor window there is a version bar showing the current version of the selected GroupValues. You can edit Major, Minor, Patch and Label directly from there. Changes are applied when you click Apply.

► Automatic version bump suggestion

When clicking Apply after making structural changes (adding or removing entries or fields), the editor will suggest a version bump automatically. You can accept the suggestion or keep the current version. It is strongly recommended to accept it so that the migration system can detect the change and update player saves correctly.